

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

Кафедра «Вычислительные системы и технологии»

ОТЧЕТ

по лабораторной работе №2

по дисциплине

Языки интернет-программирования

РУКОВОДИТЕЛЬ:

(подпись)

Яшнов М.Д.
(фамилия, и.,о.)

СТУДЕНТ:

(подпись)

Ивашина Е.Р.
(фамилия, и.,о.)

23-ИИ
(шифр группы)

Работа защищена «__» _____

С оценкой _____

Нижний Новгород
2026

Ссылка на GitHub: <https://github.com/luft15/Books-club>

Тема лабораторной работы

Реализация проекта с серверным рендерингом страниц, работа с базой данных и безопасностью.

Цель

- Освоить серверный рендеринг HTML-страниц
- Научиться безопасно работать с базой данных через чистый SQL
- Использовать архитектуру MVC и принципы DI
- Освоить контейнеризацию с Docker для приложения и БД
- Реализовать CRUD операции и взаимосвязь таблиц

1. - Все HTML-страницы должны формироваться на сервере. Клиентский JavaScript разрешён только для интерактивности

Из-за того, что основным языком программирования выбран Python, то будет использоваться фреймворк Flask

Src/controllers/index_controller.py:

```
Books-club > src > controllers > index_controller.py > ...
1  from flask import render_template
2  from app import app
3
4
5  @app.route("/")
6  def main_page():
7      return render_template("index.html")
8
9
```

Src/controllers/info_controller.py:

```
1  from flask import render_template
2  from app import app
3
4  @app.route("/info")
5  def information():
6      return render_template("information.html")
7
```

Src/controllers/plan_controller.py:

```
looks-club > src > controllers > plan_controller.py > plan
1  from flask import render_template
2  from app import app
3
4
5  @app.route("/plan")
6  def plan():
7      return render_template("plan.html")
8
```

Каждый контроллер подключается в src/app.py:

```
1  from flask import Flask
2
3  # from db.db import connect_db
4
5
6  app = Flask(__name__)
7
8  # db_con = connect_db()
9
10 from controllers.index_controller import *
11 from controllers.info_controller import *
12 from controllers.plan_controller import *
13
14 if __name__ == "__main__":
15     app.run(
16         host="0.0.0.0",
17         port=80,
18         debug=True
19     )
```

2. - Развертывание БД и приложения через Docker Compose

Созданы docker_compose:

```

Books-club > 🐳 docker-compose.yml
1  services:
2    db_booksclub:
3      image: postgres:17-alpine
4      environment:
5        POSTGRES_USER: ${DB_USER}
6        POSTGRES_DB: ${DB_NAME}
7        POSTGRES_PASSWORD: ${DB_PASSWORD}
8      ports:
9        - "${DB_PORT}:5432"
10     volumes:
11       - pgdata:/var/lib/postgresql/data
12       - ./src/database:/docker-entrypoint-initdb.d
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
15       interval: 10s
16       timeout: 5s
17       retries: 5
18     container_name: db_booksclub
19
20     backend_booksclub:
21       build:
22         context: .
23         dockerfile: Dockerfile
24       environment:
25         DB_HOST: db_booksclub
26         DB_PORT: ${DB_PORT}
27         DB_NAME: ${DB_NAME}
28         DB_USER: ${DB_USER}
29         DB_PASSWORD: ${DB_PASSWORD}
30         FLASK_ENV: development
31         FLASKDEBUG: 1
32       ports:
33         - "8080:80"
34       volumes:
35         - ./src:/app
36         - /app/__pycache__
37       depends_on:
38         db_booksclub:
39           condition: service_healthy
40       restart: on-failure
41       working_dir: /app
42       container_name: backend_flask_booksclub
43
44     volumes:
45       pgdata:

```

И создан Dockerfile:

```

Books-club > Dockerfile
1  FROM python:3.10-alpine
2
3  WORKDIR /app
4
5  COPY requirements.txt .
6  RUN pip install --no-cache-dir -r requirements.txt
7
8  COPY src/ .
9  EXPOSE 80
10
11 CMD ["python", "app.py"]

```

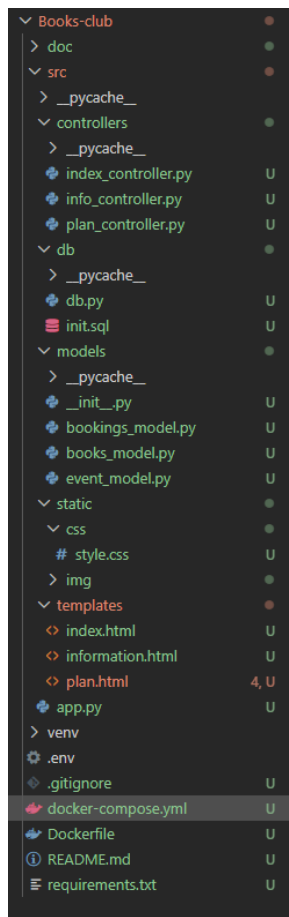
Также для нормальной работы докера нужен был файл requirements.txt. Он был создан командой `pip freeze > requirements.txt`

3. - Использование MVC архитектуры, контроллеры обрабатывают запросы, модели — работу с БД, представления — шаблоны HTML

Эта архитектура нужна чтобы отделять логику. Модели работают с базой данных — контроллеры обрабатывают запросы, валидируют их.

Представления — всё так же работают с HTML, но теперь в них есть Jinja

Архитектура:



Контроллер(главный!):

```
@app.route("/plan", methods=['GET', 'POST'])
@inject #
def plan(event_model: EventModel, db: Database):
    if request.method == "POST":
        name = request.form.get('user_name', "").strip()
        phone = request.form.get('user_phone', "").strip()
        email = request.form.get('user_email', "").strip()
        date_str = request.form.get('date', '').strip()
        time_slot = request.form.get('time_slot', "").strip()
        comment = request.form.get('comment', "").strip()
        print(f'- получены данные: {name}, {phone}, {email}, {date_str}, {time_slot}, {comment}')
```

```
def plan(event_model: EventModel, db: Database):
    try:
        booking_model = BookingModel(
            db=db,
            event_id=None,
            user_name=name,
            user_email=email,
            user_phone=phone or None,
            time_slot=time_slot or None,
            comment=comment or None
        )
        print("Объект бронирования создан")
    except ValueError as e:
        return jsonify({'status': 'error', 'errors': str(e)}), 400

    try:
        event = EventModel.get_by_date(date_str)
        if event is None:
            return jsonify({"status": "error", "errors": "Событие на эту дату не найдено!")), 404

        booking_model.event_id = event.id
    except Exception as e:
        return jsonify({"status": "error", "errors": f"ошибка при поиске события: {str(e)}")), 500

    if booking_model.save():
        print(f'- запись сохранена: {name} на {date_str} {time_slot}')
        return jsonify({"status": "ok"})

    else:
        return jsonify({"status": "error", "errors": "Не удалось сохранить бронирование")), 500
```

```

# GET-запрос
events = event_model.get_all()
events_by_date = {}
for ev in events:
    # Преобразуем дату в строку ISO
    date_val = ev['event_date']
    if hasattr(date_val, 'isoformat'):
        date_iso = date_val.isoformat()
    else:
        date_iso = str(date_val)
    book_info = f"«{ev['book_title']}» {ev['book_author']}"
    events_by_date.setdefault(date_iso, []).append(book_info)

return render_template("plan.html", services=events_by_date)

```

Модели (все написаны на чистом SQL):

Представления:

```

138         <a href="#" class="social-link">Te
139         <a href="#" class="social-link">VK
140         <a href="#" class="social-link">Yo
141     </div>
142 </div>
143 </div>
144 <div class="footer-bottom">
145     <p>&copy; 2025 BOOKSclub. Все права защище
146 </div>
147 </div>
148 </footer>
149
150 <script>
151     // События, переданные из Flask
152     const eventsByDate = {{{ services|tojson }}};
153
154     function buildMonthCalendar(year, month, eventsMap
155     const container = document.getElementById(cont
156     if (!container) return;

```

4. - CRUD операции минимум для одной сущности

```
6 class EventModel: Chat (CTRL + I) / Share (CTRL + L)
7     def validate(self) -> None:
11         raise ValueError(f"Ошибки валидации: {'', '.join(errors)}")
12
13     def save(self) -> bool:
14         """сохраняет событие (вставка или обновление)"""
15         if self.id is None:
16             return self._insert()
17         else:
18             return self._update()
19
20     def _insert(self) -> bool:
21         """вставляет новое событие и получает id через RETURNING"""
22         query = """
23             INSERT INTO events (book_id, event_date, description, max_participants)
24             VALUES (%s, %s, %s, %s)
25             RETURNING id, created_at;
26         """
27         try:
28             result = self.db.execute_query(
29                 query,
30                 (self.book_id, self.event_date, self.description, self.max_participants),
31                 fetch=True,
32             )
33             if result:
34                 self.id = result[0]["id"]
35                 self.created_at = result[0]["created_at"]
36                 return True
37             return False
38         except Exception as e:
39             print(f"Ошибка при вставке события: {e}")
40             return False
41
```



```
6 class EventModel:
0     def _insert(self) -> bool:
1         print("Ошибка при вставке события: {e}")
0         return False
1
2     def _update(self) -> bool:
3         """обновляет существующее событие"""
4         query = """
5             UPDATE events
6             SET book_id = %s, event_date = %s, description = %s, max_par
7             WHERE id = %s;
8         """
9         try:
0             self.db.execute_query(
1                 query,
2                 (self.book_id, self.event_date, self.description, self.m
3                 )
4             )
5             return True
6         except Exception as e:
7             print(f"Ошибка при обновлении события: {e}")
8             return False
9
0     def delete(self) -> bool:
1         """удаляет событие из БД"""
2         if self.id is None:
3             return False
4         query = "DELETE FROM events WHERE id = %s;"
5         try:
6             self.db.execute_query(query, (self.id,))
7             self.id = None
8             return True
9         except Exception as e:
0             print(f"Ошибка при удалении события: {e}")
1             return False
```

```
5 class EventModel: Chat (CTRL + I) / Share (CTRL + L)
6     def get_by_id(self, cls, event_id: int) -> Optional["EventModel"]:
7         query = """
8             SELECT
9                 e.id,
10                e.book_id,
11                e.event_date,
12                e.description,
13                e.max_participants,
14                e.created_at,
15                b.title AS book_title,
16                b.author AS book_author,
17                b.year AS book_year
18            FROM events e
19            JOIN books b ON e.book_id = b.id
20            WHERE e.id = %s;
21        """
22        result = self.db.execute_query(query, (event_id,), fetch=True)
23        if not result:
24            return None
25        row = result[0]
26        return cls(
27            event_id=row["id"],
28            book_id=row["book_id"],
29            event_date=row["event_date"],
30            description=row["description"],
31            max_participants=row["max_participants"],
32            created_at=row["created_at"],
33            book_title=row["book_title"],
34            book_author=row["book_author"],
35            book_year=row["book_year"],
36        )
```

```

def get_all(self) -> List["EventModel"]:
    """возвращает список всех событий с данными о книге"""
    query = """
        SELECT
            e.id,
            e.book_id,
            e.event_date,
            e.description,
            e.max_participants,
            e.created_at,
            b.title AS book_title,
            b.author AS book_author,
            b.year AS book_year
        FROM events e
        JOIN books b ON e.book_id = b.id
        ORDER BY e.event_date;
    """
    rows = self.db.execute_query(query, fetch=True)

```

```

def get_by_date(self, date_str: str) -> Optional["EventModel"]:
    """возвращает первое событие на указанную дату с данными о книге"""
    query = """
        SELECT
            e.id,
            e.book_id,
            e.event_date,
            e.description,
            e.max_participants,
            e.created_at,
            b.title AS book_title,
            b.author AS book_author,
            b.year AS book_year
        FROM events e
        JOIN books b ON e.book_id = b.id
        WHERE e.event_date = %s
        LIMIT 1;
    """
    result = self.db.execute_query(query, (date_str,), fetch=True)
    if not result:
        return None
    row = result[0]

```

```
def get_available_slots(self, event_id: int) -> int:
    """возвращает количество свободных мест на событие (0, если событие не найдено)"""
    query = """
        SELECT
            e.max_participants - COUNT(bk.id) AS available_slots
        FROM events e
        LEFT JOIN bookings bk ON e.id = bk.event_id
        WHERE e.id = %s
        GROUP BY e.id, e.max_participants;
    """
    result = self.db.execute_query(query, (event_id,), fetch=True)
    return result[0]["available_slots"] if result else 0
```

- 5. - Минимум три таблицы по предметной области. Не менее одной связи между таблицами (один-ко-многим или многие-ко-многим). Если нет идеи для предметной области — сделать таблицы для статистики действий/логов/переходов. Все таблицы должны быть в нормальных формах**

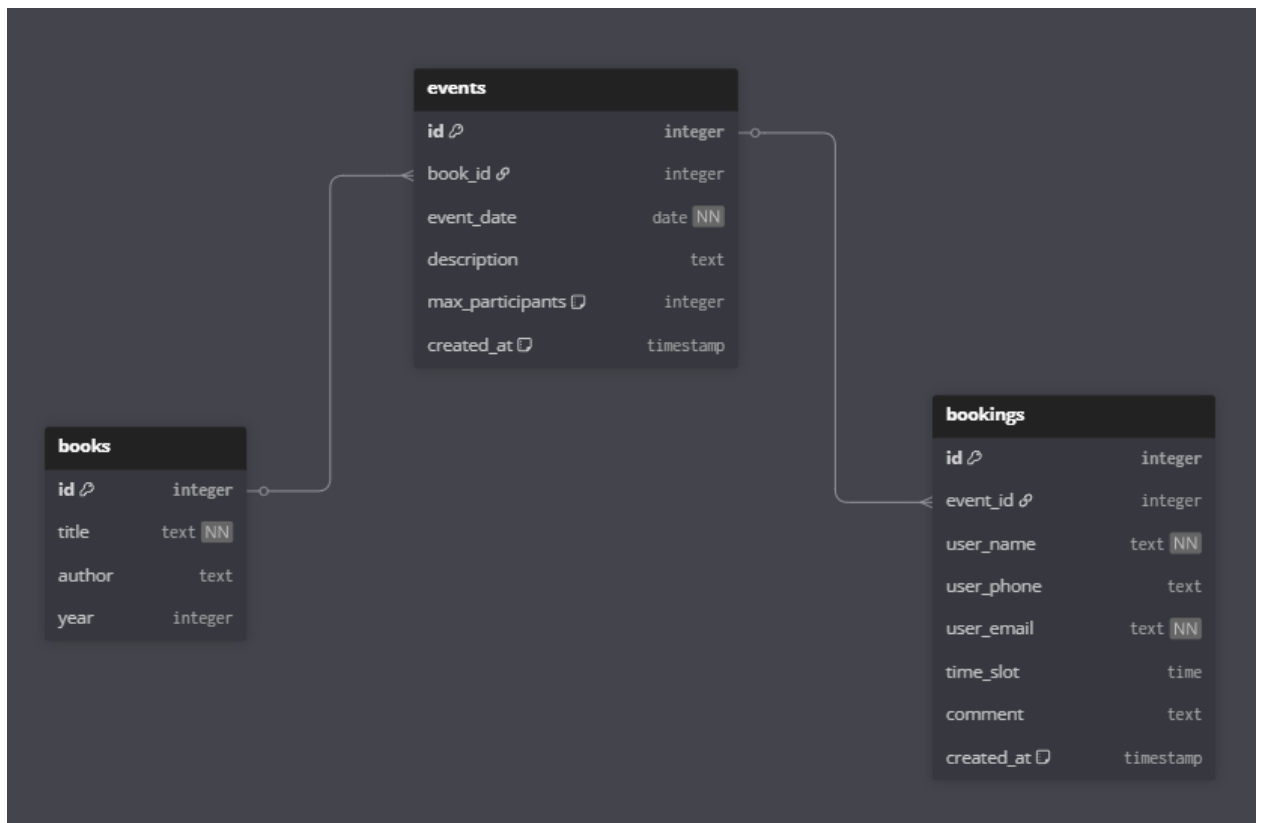
Был создан init.sql с таблицами и данными:

```

books-club > src > db > init.sql
1  -- 1. Создание таблиц
2  CREATE TABLE books (
3      id SERIAL PRIMARY KEY,
4      title TEXT NOT NULL,
5      author TEXT,
6      year INTEGER
7  );
8
9  CREATE TABLE events (
10     id SERIAL PRIMARY KEY,
11     book_id INTEGER REFERENCES books(id) ON DELETE CASCADE,
12     event_date DATE NOT NULL,
13     description TEXT,
14     max_participants INTEGER DEFAULT 20,
15     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
16 );
17
18 CREATE TABLE bookings (
19     id SERIAL PRIMARY KEY,
20     event_id INTEGER REFERENCES events(id) ON DELETE CASCADE,
21     user_name TEXT NOT NULL,
22     user_phone TEXT,
23     user_email TEXT NOT NULL,
24     time_slot TIME,
25     comment TEXT,
26     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
27 );
28
29 -- 2. Добавляем тестовые книги
30 INSERT INTO books (title, author, year) VALUES
31 ('Джейн Эйр', 'Шарлотта Бронте', 1847),
32 ('Бесы', 'Ф.М. Достоевский', 1872),
33 ('Лигея', 'Эдгар Аллан По', 1838);
34
35 -- 3. Добавляем тестовые события
36 INSERT INTO events (book_id, event_date, description, max_participants) VALUES
37 (1, '2026-03-20', 'Zoom-встреча, обсуждение «Джейн Эйр»', 30),
38 (2, '2026-02-15', 'Офлайн в клубе на ул. Книжная, 10', 20),
39 (3, '2026-03-06', 'Поэтический вечер по рассказу «Лигея»', 15);
40
41 -- 4. Добавляем тестовые записи пользователей
42 INSERT INTO bookings (event_id, user_name, user_phone, user_email, time_slot, co

```

Схема:



6. - Защита от SQL-инъекций обязательна (использование параметризованных запросов)

Она используется! Пример на скриншоте:

```

@staticmethod
def create(event_id, user_name, user_email, user_phone=None, time_slot=None, comment=None):
    query = """
        INSERT INTO bookings (
            event_id, user_name, user_phone, user_email, time_slot, comment
        ) VALUES (%s, %s, %s, %s, %s, %s);
    """
    execute_query(query, (event_id, user_name, user_phone, user_email, time_slot, comment))
  
```

7. - Обязательная защита от XSS

```

<script>
  // События, переданные из Flask
  const eventsByDate = {{{ services|tojson }}};

  function buildMonthCalendar(year, month, eventsMap, containerId) {
    const container = document.getElementById(containerId);
    if (!container) return;
    container.innerHTML = '';
  }

```

Фильтр tojson экранирует все специальные символы (включая <, >, &, ', "), превращая данные в безопасный JSON-совместимый JavaScript-код.

В любом месте, где будет написан {{ переменная }} – внутри HTML JINJA автоматически экранирует символы. Так можно предотвратить атаку на подобию "><script>alert('XSS')</script>

8. - Проверка и фильтрация входных данных

Проверка и фильтрация производится внутри модели EventModel, согласно архитектуре MVC. Чтобы код был более читаемым, была специально создана функция validate, которая возвращает ошибки, если они есть.

```

def validate(self) -> None:
    """проверяет корректность полей события"""
    errors = []

    if not isinstance(self.book_id, int) or self.book_id <= 0:
        errors.append("ID книги должен быть положительным числом")

    # проверка даты
    if not self.event_date:
        errors.append("Дата события не может быть пустой")
    else:
        try:
            # пытаемся распарсить дату в формате YYYY-MM-DD
            date.fromisoformat(self.event_date)
        except ValueError:
            errors.append("Дата должна быть в формате YYYY-MM-DD")

    if not isinstance(self.max_participants, int) or self.max_participants < 1:
        errors.append("Максимальное количество участников должно быть положительным числом")

    if self.description and len(self.description) > 500:
        errors.append("Описание не должно превышать 500 символов")

    if errors:
        raise ValueError(f"Ошибки валидации: {' '.join(errors)}")

```

Так, например, `.strip()` убирает лишние пробелы, чтобы предотвратить нежеланный ввод.

9. - Dependency Injection. Использовать DI для подключения к БД, логгера, сервисов и т.д.

Dependency Injection — это паттерн проектирования, при котором объект получает свои зависимости (другие объекты, необходимые для его работы) извне, а не создаёт их самостоятельно.

Какие зависимости были внедрены:

1. Подключение к базе данных – класс `Database`. Он отвечает за выполнение SQL-запросов к базе данных

```
class Database:
    def __init__(self):
        self.connection = None,
        self.connect()

    def connect(self):
        """Устанавливает соединение с БД, используя переменные окружения."""
        try:
            self.connection = psycopg2.connect(
                host=os.getenv('DB_HOST', 'postgres'),
                port=os.getenv('DB_PORT', '5432'),
                database=os.getenv('DB_NAME', 'postgres'),
                user=os.getenv('DB_USER', 'postgres'),
                password=os.getenv('DB_PASSWORD', 'postgres')
            )
            print("Успешное подключение к базе данных")
        except Exception as e:
            print(f"Ошибка подключения к БД: {e}")
            raise # Прерываем запуск приложения, если БД недоступна
```



```

def execute_query(self, query, params=None, fetch=False):
    """
    Универсальная функция для выполнения SQL-запросов.
    """
    if not self.connection or self.connection.closed:
        self.connect()

    try:
        with self.connection.cursor(cursor_factory=RealDictCursor) as cursor:
            cursor.execute(query, params)
            if fetch:
                result = cursor.fetchall()
            else:
                result = None
            self.connection.commit()
            return result
    except Exception as e:
        self.connection.rollback()
        print(f'ошибка выполнения запроса {e}\nЗапрос: {query}\n Params:{params}')
        raise

def close(self):
    '''закрывает соединение с БД'''
    if self.connection and not self.connection.closed:
        self.connection.close()
        print('соединение с БД закрыто')

```

2. Модель для работы с событиями – EventModel. Точнее, она использует экземпляр класса Database

```

class EventModel:
    """модель события"""

    def __init__(
        self,
        db: Database,
        book_id: Optional[int] = None,
        event_date: Optional[str] = None,

```

Соответственно все запросы к БД выполняются в виде `self.db.execute_query` – метод класса `Database`

```
def delete(self) -> bool:
    """удаляет событие из БД"""
    if self.id is None:
        return False
    query = "DELETE FROM events WHERE id = %s;"
    try:
        self.db.execute_query(query, (self.id,))
        self.id = None
        return True
    except Exception as e:
        print(f"Ошибка при удалении события: {e}")
        return False
```

3. Конфигурация: модуль AppModule

Файл `src/dependencies.py`:

```
from injector import Module, provider, singleton
from db.db import Database
from models.bookings_model import BookingModel
from models.event_model import EventModel

# AppModule – это инструкция для DI-контейнера
# о том, как создавать зависимости.
class AppModule(Module):
    @provider
    @singleton
    def provide_database(self) -> Database:
        return Database()

    # @provider
    # @singleton
    # def provide_booking_model(self, db: Database) -> BookingModel:
    #     return BookingModel(db)

    @provider
    @singleton
    def provide_event_model(self, db: Database) -> EventModel:
        return EventModel(db)
```

- `@provider` указывает, что метод предоставляет зависимость.

- `@singleton` гарантирует, что на всё время работы приложения будет создан только один экземпляр (например, одно подключение к БД).
- `Injector` автоматически передаёт `db` в `provide_event_model`, потому что знает, как создать `Database`.

4. Инициализация в приложении (`app.py`). Важно было инициализировать `Flask-Injector` ПОСЛЕ всех контроллеров:

```
Books-club > src > app.py > ...
1  from flask import Flask
2  from dependencies import AppModule
3  from flask_injector import FlaskInjector
4
5  app = Flask(__name__)
6
7  from controllers.index_controller import *
8  from controllers.info_controller import *
9  from controllers.plan_controller import *
10
11  Flask.url_for.__annotations__ = {} # нужно, чтобы Flask не оборачивал url_for в представлениях
12  FlaskInjector(app=app, modules=[AppModule()])
13
14  if __name__ == "__main__":
15      app.run(
16          host="0.0.0.0",
17          port=80,
18          debug=True
19      )
```

5. В контроллере `plan(...)` я использовала декоратор `@inject`, чтобы `flask-injector` автоматически представил готовые объекты при вызове. Также в функцию нужно было передать необходимые зависимости как аргументы функции

```
@app.route("/plan", methods=['GET', 'POST'])
@inject #
def plan(event_model: EventModel, db: Database):
    if request.method == "POST":
        name = request.form.get('user_name', "").strip()
        phone = request.form.get('user_phone', "").strip()
        email = request.form.get('user_email', "").strip()
        date_str = request.form.get('date', '').strip()
        time_slot = request.form.get('time_slot', "").strip()
        comment = request.form.get('comment', "").strip()
        print(f'- получены данные: {name}, {phone}, {email}, {date_str}, {time_slot}, {comment}')
```

10.- Все параметры (строки подключения к БД, ключи) хранятся в переменных окружения или конфигурационных файлах

```
oks-club > ⚙ .env
1  DB_NAME=db_name
2  DB_USER=db_user
3  DB_PASSWORD=db_password
4  DB_PORT=5432
5  |
```

Также этот файл был добавлен в .gitignore, чтобы случайно не запустить его.